DOKUMENTACJA PROJEKTOWA

Aplikacja służąca do rozpoznawania ubytków w drogach na podstawie pary zdjęć z kamery stereoskopowej

Organizacja i infrastruktura projektu

1 Opis projektu

1.1 Nazwa projektu

„Aplikacja służąca do rozpoznawania ubytków w drogach na podstawie pary zdjęć z kamery stereoskopowej” (ang. Application for recognizing bumps in roads on the basis of a pair of images from a stereo camera).

1.2 Adresowany problem

Obecne systemy wizyjne oparte na płaskich obrazach 2D mają problemy z przestrzenną oceną uszkodzeń drogi. Zjawisko to, nazywane "dwuznacznością tekstury", sprawia, że system 2D może pomylić głęboką wyrwę z niegroźną plamą oleju, cieniem lub kałużą, co generuje fałszywe alarmy i stanowi zagrożenie dla pojazdów. Z kolei mniej omylne komercyjne systemy 3D (takie jak skanery laserowe LiDAR na pojazdach inspekcyjnych) są ekstremalnie drogie i zbyt duże, aby stosować je masowo w autach konsumenckich.

1.3 Obszar zastosowania i rynek

Branża motoryzacyjna (pojazdy autonomiczne, systemy wspomagania kierowcy ADAS) oraz sektor zarządzania infrastrukturą drogową.

1.4 Interesariusze, użytkownicy i ich potrzeby

- **Kierowcy pojazdów.** Stanowią kluczowych interesariuszy oraz bezpośrednich użytkowników systemu w kontekście jego integracji z systemami ADAS. Ich podstawową potrzebą jest otrzymywanie szybkich i wiarygodnych ostrzeżeń o rzeczywistych zagrożeniach na drodze (takich jak ubytki nawierzchni), przy jednoczesnym ograniczeniu liczby fałszywych alarmów wynikających z błędnej interpretacji obrazu (np. cieni, plam czy kałuż). System powinien wspierać kierowcę w podejmowaniu decyzji poprzez zwiększenie bezpieczeństwa jazdy, ograniczenie ryzyka uszkodzenia pojazdu oraz poprawę komfortu prowadzenia.

- **Systemy ADAS i komputery pokładowe pojazdów autonomicznych.** Takie systemy potrzebują otrzymywać w czasie rzeczywistym szybkie i wiarygodne dane o trójwymiarowych, fizycznych wymiarach przeszkody, aby podjąć ułamkowosekundową decyzję o hamowaniu awaryjnym lub ominięciu ubytku.

- **Zarządcy dróg.** Pracownicy zajmujący się infrastrukturą drogową potrzebują cyklicznego, zautomatyzowanego monitorowania dróg (np. za pomocą kamer montowanych w autobusach miejskich). Oni oczekują raportów zawierających nie tylko lokalizację, ale też dokładną objętość ubytku, co pozwoli im oszacować ilość materiału (np. asfaltu) potrzebnego do naprawy i obniżyć koszty inspekcji.

1.5 Cel i zakres produktu

Głównym celem jest zbudowanie w języku Python modułu percepcyjnego opartego na sztucznej inteligencji, który przestrzennie przeanalizuje uszkodzenia jezdni i dostarczy dane o ich wielkości. Zakres projektu obejmuje również aspekt sprzętowy: zaprojektowanie i budowę taniej, autorskiej kamery stereoskopowej (z użyciem kamerek internetowych i minikomputera Raspberry Pi), a także samodzielną akwizycję danych do uczenia algorytmu. Oprogramowanie zostanie udostępnione jako open-source na licencji MIT.

1.6 Ograniczenia

Rozwiązanie opiera się na pasywnej rekonstrukcji stereoskopowej, co oznacza, że wymaga światła zastanego i nie zadziała w całkowitych ciemnościach. Dodatkowo system może mieć spadek skuteczności na drogach o jednolitej, "płaskiej"teksturze bez punktów odniesienia. Istnieje również wrażliwość na drgania stelaża podczas jazdy, które mogą zaburzyć kalibrację.

1.7 Inne współpracujące systemy

Aplikacja ma działać równolegle jako niezależne, dodatkowe źródło danych dla istniejących systemów ADAS oraz głównych komputerów pokładowych (odpowiadających np. za systemy trajektorii pojazdu).

1.8 Termin i główne etapy projektu

- Faza badawcza (marzec - kwiecień): przegląd literatury i projekt układu kamer.
- Faza sprzętowa (kwiecień - maj): montaż platformy stereoskopowej, skrypt synchronizacji i kalibracja.
- Akwizycja danych (maj - lipiec): zebranie zsynchronizowanych par zdjęć dróg w terenie.
- Faza programistyczna (lipiec - wrzesień): implementacja algorytmów analizy obrazu i wyliczania objętości.
- Faza testów i dokumentacji (wrzesień - listopad): weryfikacja dokładności, sporządzenie dokumentacji i publikacja kodu (MIT).

2 Interesariusze i użytkownicy

Interesariuszy projektu można podzielić na trzy główne grupy: bezpośrednich użytkowników końcowych oprogramowania, interesariuszy pośrednich (korzystających z efektów działania systemu) oraz interesariuszy projektowych (związanych z procesem akademickim).

2.1 Użytkownicy końcowi

- **Komputery pokładowe pojazdów autonomicznych i systemy ADAS.** Są to główne systemy bezpośrednio konsumujące dane z aplikacji. Wymagają one dostarczania w czasie rzeczywistym precyzyjnych wektorów i parametrów geometrycznych ubytku (maksymalna głębokość), aby w ułamku sekundy podjąć decyzję o hamowaniu awaryjnym, bezpiecznym ominięciu przeszkody lub ostrzeżeniu kierowcy.
- **Zarządcy infrastruktury drogowej.** Podmioty odpowiedzialne za utrzymanie dróg, wykorzystujące system do zautomatyzowanego monitoringu stanu nawierzchni (np. instalując platformy stereoskopowe na autobusach miejskich). Oczekują oni automatycznych raportów o lokalizacji, głębokości i dokładnej objętości ubytku, co pozwala im zoptymalizować proces naprawczy (np. wyliczyć, ile asfaltu potrzeba na łatę) i drastycznie obniżyć koszty w porównaniu do komercyjnych skanerów LiDAR.
- **Kierowcy i pasażerowie pojazdów.** Ostateczni beneficjenci systemu (w tym systemów wczesnego ostrzegania ADAS). Zyskują wyższe bezpieczeństwo oraz ochronę zawieszenia pojazdu, dzięki temu, że system przestrzenny 3D nie ulega "oszustwom wizualnym"(takim jak plamy oleju czy cienie drzew) i reaguje tylko na faktyczne, fizyczne zagrożenia na drodze.

2.2 Interesariusze pośredni

- **Społeczność badawcza i inżynierowie open-source.** Badacze z branży motoryzacyjnej, computer vision oraz programiści sztucznej inteligencji. Ponieważ moduł percepcyjny zostanie w całości napisany w języku Python i udostępniony bezpłatnie na otwartej licencji MIT, ta grupa będzie mogła swobodnie analizować, rozwijać i integrować kod we własnych rozwiązaniach badawczych.

2.3 Interesariusze projektowi (akademiccy)

- **Promotor projektu.** Jego potrzebą jest terminowa realizacja projektu zgodnie z celami inżynierskimi, weryfikacja poprawności przyjętej metodologii badawczej oraz ocena gotowej pracy dyplomowej.
- **Katedra.** Jednostka uczelni sprawująca formalny nadzór nad realizacją pracy dyplomowej.

3 Zespół

3.1 Skład zespołu

W skład dwuosobowego zespołu wchodzą studenci Politechniki Gdańskiej:

- Taras Shuliakevych (katedra: KAiMS).
- Bartosz Lewczuk (katedra: KISI).

3.2 Umiejętności członków zespołu

Zespół ma umiejętności interdyscyplinarne i wielofunkcyjne, co pozwala na samodzielną realizację wszystkich przyrostów projektu (od sprzętu po AI). Zespół jako całość posiada kompetencje w zakresie:

- Programowania w języku Python.
- Budowy i obsługi architektur głębokiego uczenia w PyTorch (np. YOLO, IGEV-Lite).
- Przetwarzania obrazu i kalibracji kamer przy użyciu biblioteki OpenCV.
- Implementacji algorytmów matematycznych i operacji macierzowych (NumPy, SciPy).
- Inżynierii sprzętowej (projektowanie stelaża, praca z mikrokontrolerami np. Raspberry Pi).
- Obsługi systemu kontroli wersji Git.

3.3 Obszary odpowiedzialności

Projekt realizowany jest w oparciu o zwinne metodyki wytwarzania oprogramowania. Zespół funkcjonuje jako samoorganizujący się zespół deweloperski który ponosi odpowiedzialność za działający produkt końcowy. Zamiast sztywnego przypisania ról na cały czas trwania projektu, zadania są dobierane dynamicznie z rejestru w zależności od celów aktualnego etapu prac. Wszelkie kluczowe decyzje architektoniczne, wybór algorytmów oraz integracja poszczególnych modułów – na przykład połączenie sprzętowego wyzwalania kamer z wejściem do sieci neuronowej – realizowane są wspólnie. Obaj członkowie zespołu wspólnie dbają o najwyższą jakość kodu oraz terminowe sporządzanie dokumentacji.

3.4 Organizacja pracy

Projekt realizowany jest w modelu hybrydowym:

- **Praca w jednym miejscu.** Wymagana podczas fazy sprzętowej (fizyczny montaż stelaża i kamer, kalibracja w warunkach kontrolowanych) oraz w fazie akwizycji danych (wspólne wyjazdy w teren z platformą pomiarową w celu zebrania zsynchronizowanych par zdjęć z dróg).
- **Praca w rozproszeniu (zdalnie).** Faza badawcza (przegląd literatury), faza programistyczna (implementacja algorytmów AI, całkowania i segmentacji) oraz pisanie dokumentacji będą realizowane w rozproszeniu, przy wykorzystaniu współdzielonego środowiska (np. repozytorium GitHub do wymiany kodu).

3.5 Dane kontaktowe

Imię i nazwisko	**Adres e-mail**
Taras Shuliakevych	s196615@student.pg.edu.pl
Bartosz Lewczuk	s193267@student.pg.edu.pl

4 Komunikacja w zespole i z interesariuszami

- **Komunikacja w zespole.** Opiera się na ciągłej i bezpośredniej wymianie informacji, co pozwala na szybkie rozwiązywanie problemów pojawiających się na styku sprzętu i oprogramowania. Jak wcześniej już wspomniano, współpraca realizowana jest w modelu hybrydowym. Podstawą synchronizacji są krótkie, zdalne spotkania, podczas których obaj członkowie zespołu omawiają zrealizowane zadania, plany na dany dzień oraz ewentualne blokady technologiczne. Do bieżącej, asynchronicznej komunikacji zdalnej wykorzystywane są Discord i Messenger, natomiast dyskusje ściśle programistyczne i architektoniczne toczą się wokół kodu bezpośrednio w repozytorium GitHub (m.in. poprzez komentarze do Pull Requestów i wzajemne Code Review). Praca stacjonarna i bezpośrednia komunikacja twarzą w twarz wymuszane są z kolei przez fazy sprzętowe – zespół spotyka się fizycznie podczas projektowania i montażu autorskiego stelaża z kamerami stereoskopowymi, synchronizacji mikrokontrolera Raspberry Pi, a także podczas wspólnych wyjazdów w teren w celu akwizycji zsynchronizowanych par zdjęć z dróg.
- **Komunikacja z otoczeniem projektu.** Ma charakter ustrukturyzowany. Spotkania z promotorem odbywają się cyklicznie (co dwa tygodnie) w formie zdalnej za pośrednictwem platformy MS Teams. Celem tych konsultacji nie jest jedynie teoretyczne omawianie postępów, lecz prezentacja kolejnych, działających przyrostów.

5 Współdzielenie dokumentów i kodu

Centralnym punktem wymiany wszelkich artefaktów – zarówno kodu źródłowego, jak i dokumentacji – jest zdalne repozytorium oparte na systemie kontroli wersji Git. Ze względu na to, że gotowy moduł percepcyjny ma zostać udostępniony jako projekt open-source, platformą docelową dla repozytorium jest GitHub. Obaj członkowie zespołu deweloperskiego posiadają do niego pełny dostęp, a proces integracji nowych przyrostów opiera się na mechanizmie Pull Requestów oraz wzajemnym Code Review. Aby zapewnić sprawną organizację i uniknąć chaosu informacyjnego, przyjęto następujące zasady zarządzania infrastrukturą:

- **Utrzymanie repozytorium i kod.** Główną odpowiedzialność za założenie repozytorium na platformie GitHub, konfigurację głównych gałęzi (np. main, develop) oraz pilnowanie licencji MIT ponosi Bartosz Lewczuk.

  Adres repozytorium to: `https://github.com/GizaBartosz/stereo-bump-detection`

- **Porządek w dokumentacji.** Za przejrzystą strukturę katalogów z dokumentacją, aktualizację pliku README oraz pilnowanie spójności raportów z kolejnych etapów prac odpowiada Taras Shuliakevych.

- **Wersjonowanie.** Zarówno kod, jak i dokumentacja projektowa wersjonowane są w pełni automatycznie za pomocą systemu Git w repozytorium.

- **lista zadań.** Bieżące zadania, błędy oraz priorytety prac zarządzane są za pomocą narzędzia GitHub Issues. Pozwala to na przejrzyste przypisywanie odpowiedzialności i śledzenie postępów w czasie rzeczywistym.

- **Schemat nazewnictwa plików.** W przypadku zewnętrznych plików binarnych, raportów wyeksportowanych do formatu PDF czy schematów, obowiązuje jednolity schemat nazewnictwa bez użycia spacji, ułatwiający wyszukiwanie:
  NazwaDokumentu_vX.Y.Z_RRRR-MM-DD.rozszerzenie.

- **Szablon dokumentu projektu.** Wszelkie oficjalne raporty tworzone są w oparciu o ujednolicony szablon z zachowaniem spójnej identyfikacji wizualnej. Każdy taki dokument posiada czytelną stronę tytułową, a w nagłówkach i stopkach zawiera najważniejsze dane identyfikacyjne: skróconą nazwę projektu, datę utworzenia dokumentu, typ dokumentu oraz autorów.

Dodatkowo zebrane zdjęcia przechowywane są w chmurze Google Drive. Adres: `https://drive.google.com/drive/folders/1ASLOtvnG603Y_5_YFWGnfdNoMSS-rU1v?usp=sharing`

6 Narzędzia

W celu zapewnienia sprawnej organizacji pracy, płynnego tworzenia dokumentacji oraz odpowiedniej wydajności samego oprogramowania, zespół wykorzystuje następujące grupy narzędzi:

6.1 Tworzenie dokumentacji i komunikacja

- **Google Docs i Google Slides.** Wykorzystywane do bieżącej, synchronicznej współpracy przy tworzeniu szkiców raportów, notatek ze spotkań oraz przygotowywania prezentacji seminaryjnych.

- **Overleaf.** Internetowy edytor LaTeX, w którym docelowo powstawać będą zaawansowane raporty techniczne oraz ostateczny dokument pracy dyplomowej.

- **GitHub.** Poza kontrolą wersji kodu, repozytorium na platformie GitHub pełni kluczową rolę w wersjonowaniu plików tekstowych i prowadzeniu technicznej dokumentacji projektu (m.in. poprzez rozbudowane pliki README), przygotowując grunt pod publikację open-source.

- **Google Drive.** Służy jako przestrzeń w chmurze do przechowywania zasobów nietekstowych. Jest wykorzystywany głównie do archiwizacji bazy danych (zbioru par zdjęć stereoskopowych) oraz dużych plików binarnych i wag modeli, które ze względu na rozmiar nie powinny obciążać repozytorium Git.

6.2 Środowisko wytwórcze i zarządzanie kodem

- Całość modułu percepcyjnego zaimplementowana zostanie w języku Python 3.
- Główne zintegrowane środowiska programistyczne (IDE) wykorzystywane przez zespół to PyCharm oraz Visual Studio Code, wspomagane przez menedżer środowisk wirtualnych Conda.
- Podstawą ciągłej integracji i rozproszonej pracy nad kodem jest system Git, zsynchronizowany ze wspomnianym już repozytorium GitHub.

6.3 Modelowanie AI i przetwarzanie obrazu

- **OpenCV.** Używane w początkowej fazie działania systemu do wstępnego przetwarzania obrazu, kalibracji obiektywów w oparciu o wzorzec szachownicy oraz wykonania rektyfikacji epipolarnej zsynchronizowanych zdjęć.
- **PyTorch.** Główne środowisko pracy dla modeli sztucznej inteligencji. Posłuży do budowy i obsługi architektur głębokiego uczenia, w tym detektora YOLO (odpowiedzialnego za wycięcie regionu zainteresowania – ROI) oraz sieci IGEV-Lite (generującej mapy dysparycji).
- **NumPy i SciPy.** Biblioteki niezbędne w końcowej fazie analizy 3D. Wykorzystywane do operacji macierzowych, transformacji współrzędnych przestrzennych, implementacji algorytmu RANSAC (do szukania płaszczyzny referencyjnej drogi) oraz matematycznego całkowania objętości fizycznej ubytku.

6.4 Infrastruktura badawcza i sprzętowa

- **Autorska platforma stereoskopowa.** Stanowisko pomiarowe składające się ze specjalnie zaprojektowanego stelaża i zestawu zsynchronizowanych kamer (np. webkamer).
- **Raspberry Pi.** Mikrokontroler odpowiedzialny za sprzętową obsługę kamer – w tym m.in. za precyzyjne wyzwalanie klatek, by zminimalizować błędy w synchronizacji.
- **Jednostka GPU.** Komputer z wydajnym układem graficznym, który jest niezbędny do przeprowadzania uciążliwych obliczeniowo operacji generowania map dysparycji, przetwarzania gęstych chmur punktów i uruchamiania sieci neuronowych.